PRODUCT AND ENGINEERING DELIVERY CONTRACT

AssetPulse Lite

Real-time asset monitoring and maintenance workflow

Deliver one production-style vertical journey: synthetic telemetry is accepted safely, processed durably, shown as a live alert, and converted into assigned maintenance work. Scope outside this contract is not part of v1.0.

Decision	Current contract
Status	Approved build contract - zero-cost delivery revision
Architecture	Java 21 / Spring modular monolith, React / TypeScript, PostgreSQL
Release	Public HTTPS portfolio demo on services that require no payment method and no paid subscription
Delivery target	Render Free composite web service plus Neon Free PostgreSQL
Remaining checkpoints	v0.6 product assurance, v0.7 zero-cost public operations, then v1.0 release
Companion	AssetPulse Lite - Eight-Week Delivery and Git Plan v1.1

A requirement is complete only when its behavior, tests, and delivery evidence can be inspected.

Version 1.1 | 31 August 2026

01 | CONTRACT USE

Contract use

One controlling product scope with a strict cost boundary.

- This contract controls v1.0 scope, engineering requirements, evidence, and acceptance.
- The companion delivery plan controls sequence, checkpoints, and repository workflow; it cannot expand scope.
- Earlier specifications are background only when they conflict with this contract.
- New capability enters v1.0 only when it closes an existing acceptance criterion.
- Public hosting must remain usable without entering payment details, starting a paid trial, or accepting automatic overage billing.

Product objective

AssetPulse Lite helps an operations team detect an equipment problem from synthetic telemetry and turn the resulting alert into assigned, auditable maintenance work.

Required users

Role	Permitted responsibility	Required denial
Operations Admin	View operations, acknowledge alerts, create and assign work.	Cannot access another organisation.
Technician	View assigned work; start and complete own work.	Cannot assign work or mutate another technician's work.
Viewer	Read dashboards, assets, alerts, and work orders.	Every mutation is rejected by the backend.

Ninety-second acceptance journey

- Open the public demo and sign in to the seeded demonstration organisation.
- Start the Overheating Pump simulator scenario.
- Observe telemetry, a threshold breach, and one live alert without refreshing.
- Acknowledge the alert, create a work order, and assign the seeded technician.
- Switch to the technician account, start the work, and mark it done.
- Return to the dashboard and inspect the consistent activity and audit trail.

02 | SCOPE

Scope

Engineering depth is protected by reducing configuration, administration, and infrastructure breadth.

Required capabilities

- Identity: seeded session login, two organisations, and three roles.
- Domain: seeded assets, sensors, and one threshold rule; read-only configuration UI.
- Telemetry: atomic, idempotent batches of 1-100 readings and bounded reads.
- Simulator: normal and overheating scenarios through the public API only.
- Alerts: durable evaluation, fingerprinting, cooldown, history, and SSE refresh.
- Work: create from alert, assign, progress through four states, and recover from optimistic conflict.
- Delivery: Docker Compose, CI, free public HTTPS hosting, health checks, smoke, and rollback guidance.
- Operations: structured logs, four metrics, one trace, one measured load scenario, and dependency / secret / image scans.
- Evidence: automated tests, current documentation, demonstration media, and tagged releases.

Explicitly excluded before v1.0

- Registration, invitations, password reset, user administration, and organisation switching.
- Asset, sensor, or threshold-rule create / edit screens.
- Additional simulator scenarios, real equipment, external notifications, billing, uploads, maps, chat, or mobile applications.
- Extra work-order states, advanced analytics, complex filters, multi-region claims, microservices, messaging clusters, or orchestration platforms.
- External identity providers and broad infrastructure-as-code coverage.
- Any host that requires a paid subscription, payment card, paid trial, or automatic overage commitment.

Current checkpoint decision

v0.5 is complete. Its product gate is supported by merged green GitHub-hosted CI and exact-source local packaged, browser, database, and conflict evidence. No public deployment is claimed for v0.5. Public zero-cost deployment is deliberately a v0.7 outcome. Only v0.6, v0.7, and v1.0 remain.

03 | FUNCTIONAL REQUIREMENTS

Functional requirements

Stable identifiers are used in issues, tests, pull requests, and release evidence.

Identity, configuration, and telemetry

ID	Requirement	Acceptance evidence
AUTH-01	Seeded users sign in and out through server-managed browser authentication.	Protected requests fail after logout; credentials are not logged.
AUTH-02	Organisation and role come only from trusted authentication context.	Body, query, or header identifiers cannot alter scope.
AUTH-03	Every tenant-owned query, mutation, and event stream is organisation-scoped.	Positive and negative integration evidence uses two organisations.
AUTH-04	Backend enforces the three-role matrix and technician ownership.	Direct forbidden requests return non-leaking responses.
CFG-01	Seed two organisations and at least two monitored assets in the demo organisation.	A clean database produces deterministic isolated data.
CFG-02	Each relevant asset has a sensor and one temperature rule.	Constraints prevent cross-organisation relationships.
CFG-03	Authorised users read configuration; v1.0 has no edit surface.	Tenant-safe reads exist and administration routes do not.
TEL-01	Accept 1-100 readings with an organisation-scoped idempotency key.	Exact retry returns the original result without duplicates.
TEL-02	Validate sensor, decimal value, and time; store the batch atomically.	One invalid reading rolls back the entire batch.
TEL-03	Commit readings and one durable processing event in one transaction.	Fault injection proves neither side can exist alone.
TEL-04	Expose a bounded tenant-safe sensor time-range query.	An index supports the query and foreign data is never returned.
TEL-05	Provide normal and overheating scenarios through the public API.	Both run from documented commands without database access.

Durable alerts, work orders, and audit

ID	Requirement	Acceptance evidence
EVT-01	A PostgreSQL worker safely claims durable pending events and completes domain effects after request commit.	Restart resumes work and concurrent workers do not duplicate effects.
EVT-02	Failures retry with bounded backoff and enter a durable dead state.	Attempts and safe errors are visible; protected retry exists.
ALR-01	A deterministic breach opens or updates one alert fingerprint.	Repeated readings inside cooldown update occurrence count.
ALR-02	Alerts use OPEN, ACKNOWLEDGED, and RESOLVED states.	Illegal transitions return 409 and create no history.
ALR-03	Committed alert changes reach authorised clients by SSE.	Client refreshes authoritative state after updates and reconnects.
WO-01	Operations Admin creates from an alert and assigns a technician.	Alert link is immutable and assignment is audited.
WO-02	State is OPEN -> ASSIGNED -> IN_PROGRESS -> DONE.	Server rejects skipped and reversed transitions.
WO-03	Only the assigned technician starts and completes work.	Role and ownership tests cover permit and deny paths.
WO-04	Assignment and transitions append immutable chronological history.	Rejected commands append no history or audit record.
WO-05	Mutations require the expected version.	Two stale clients yield one success and one recoverable 409.
AUD-01	Authentication and admitted commands create scoped audit events.	Actor, action, subject, time, and correlation ID are visible without credentials or raw telemetry.

04 | PRODUCT SURFACE AND API

Product surface and API

The interface exposes system behavior clearly; it is not a separate design-system project.

Surface	Required behavior
Landing and login	Product statement, live status, seeded account guidance, and sign-in.
Dashboard	Asset count, open-alert count, active-work count, recent activity, and simulator launch.
Asset detail	Read-only sensor / rule summary plus chart and accessible table fallback.
Alert queue and detail	Live list, acknowledge / resolve, and create-work-order action.
Work orders	List / detail, assignment, role-correct transitions, history, and conflict recovery.
Audit and operations	Bounded audit trail, dead-processing state, and protected retry.

Required interface states

- Useful loading, empty, unavailable, forbidden, and stale-conflict states.
- Backend denial remains mandatory even when a control is hidden.
- Status is not communicated by colour alone; primary flows are keyboard usable.
- The layout works at one practical desktop width and one practical mobile width.

API contract

- Versioned REST routes and published OpenAPI documentation.
- Stable RFC-style problem responses with code, message, field errors, and correlation ID.
- Bounded batches, lists, and time ranges with documented defaults and maxima.
- Compile-time checked frontend request and response types.

Runtime topology

Component	Responsibility	Boundary
React client	Product screens, server state, and SSE subscription.	Calls same-origin application routes only.
Nginx edge	Static assets, same-origin API proxy, and stream-safe proxy settings.	Public listener inside the composite image.
Spring Boot	Authentication, tenancy, transactions, rules, alerts, work, audit, and worker.	Loopback-only listener; owns all business rules.
Neon Free PostgreSQL	Business data, telemetry, durable events, history, and audit.	TLS source of truth; app uses a least-privilege role.
Simulator	Deterministic public-API telemetry.	Untrusted client with no database access.

05 | ENGINEERING CONTRACT

Engineering contract

Correctness must not depend on a browser control, in-memory queue, or single process lifetime.

Architecture rules

- Use a modular monolith organised by product feature; do not create separate business services.
- Controllers own transport only; application services own behavior and transaction boundaries.
- Explicit API models remain separate from persistence rows.
- Telemetry plus processing intent, and each work transition plus history / audit, are atomic.
- Post-commit notification failure cannot roll back completed domain work.
- Prefer readable explicit code over speculative abstraction.

Security and reliability controls

Risk / invariant	Required control and verification
Cross-tenant access	Trusted organisation context, scoped SQL, relational constraints, and two-organisation negative integration tests.
Broken role or ownership	Backend role / assignee enforcement and a complete permission matrix.
Session and CSRF	Secure HttpOnly cookie, bounded lifetime, logout, mutation CSRF, and rejection tests.
Secret exposure	No committed secrets; dependency, secret, and image scans before release.
Telemetry retry	Exact retry creates no duplicate readings or downstream effects; mixed invalid batches commit nothing.
Durable processing	Restart resumes pending work; concurrent or stale claims cannot duplicate or corrupt effects.
Alert recurrence	Fingerprint, cooldown, and database constraints retain at most one active alert per scope.
Work conflict	Expected-version update yields one winner and an explicit recoverable conflict for stale clients.
Free-host lifecycle	Sleep or redeploy may invalidate process-local sessions; the client must recover honestly and all durable state remains in PostgreSQL.

Zero-cost delivery boundary

- One Render Free Docker web service runs Nginx and Spring together, preserving same-origin cookies, CSRF, and SSE.
- Neon Free PostgreSQL supplies the durable store; the app receives only a least-privilege TLS connection.
- No database owner credential, payment method, or deploy token is committed to GitHub.
- The service must fit 0.1 CPU and 512 MB RAM. JVM heap, database pool, and worker frequency are bounded and measured before v0.7 closes.
- Free-service suspension, cold starts, ephemeral application storage, quota changes, and loss of process-local sessions are release limitations, not hidden scale claims.

06 | VERIFICATION AND EVIDENCE

Verification and evidence

Test counts are not targets; coverage is judged by critical behavior and failure paths.

Layer	Required focus	Release evidence
Domain unit	Threshold, cooldown, transitions, ownership, and retry schedule.	Fast deterministic suite.
Backend integration	Flyway, constraints, tenancy, RBAC, idempotency, atomic processing, retry, and optimistic locking.	PostgreSQL-backed suite on every pull request.
Frontend	Typed transforms, UI states, permissions, and conflict recovery.	Focused component and API tests.
End-to-end	Live incident, isolation / role, and concurrency conflict.	Three local Playwright journeys; live incident against the v0.7 deployment.
Security	Session / CSRF, matrix, validation, headers, secrets, and redaction.	Named tests and scanner reports.
Performance	Bounded ingestion and telemetry-to-alert processing.	One k6 report and one inspected query plan.
Delivery	Composite image, free-host config, health, smoke, identity, and rollback.	Identifiable image / commit and repeatable public verifier.

Required journeys

- Live incident: overheating -> alert -> acknowledge -> order -> assign -> technician done; no manual refresh and consistent audit relationships.

- Isolation and role: Viewer mutation and direct foreign-resource request fail without leaked names, counts, or events.

- Concurrency: two clients mutate one work-order version; one succeeds and the stale client follows the explicit refresh path without replay.

Operational evidence

- Structured logs include request / correlation ID and safe actor / organisation context.

- At least four useful metrics cover request latency / error, pending events, processing lag, and retry / dead state.

- One trace follows telemetry acceptance through durable processing to alert creation.

- The k6 command, raw summary, environment, and limitations are published.

- Deployment identification, smoke, cold-start expectations, and previous-image rollback are documented.

07 | DELIVERY AND DEFINITION OF DONE

Delivery and definition of done

The repository is the primary acceptance package and must describe the implemented release truthfully.

Required repository artifacts

Artifact	Required contents
backend/	Spring Boot application, migrations, configuration, and automated tests.
frontend/	React / TypeScript product, API types, focused tests, and Nginx configuration.
simulator/	Normal and overheating clients plus documented commands.
infra/	Dockerfiles, Compose support, public smoke, database bootstrap, and zero-cost hosting image.
render.yaml	One free web service, CI-gated deployment, health path, and manually supplied secrets.
docs/	Architecture, decisions, security, operations, performance, evidence, and current issue records.
README	Pitch, status, free-host plan / link, quick start, architecture, evidence, and limitations.

Release checklist

1. Public HTTPS demo, status endpoint, and seeded login work on the no-card free plan.
2. A clean clone starts backend, frontend, and PostgreSQL through documented commands.
3. Two organisations and three roles are seeded; tenant and permission denials pass.
4. Normal and overheating scenarios submit bounded public-API telemetry.
5. Telemetry is atomic and idempotent; bounded range query and chart / table work.
6. Durable processing survives retry / restart and exposes safe dead-state recovery without duplicate effects.
7. Threshold breach produces one live alert with fingerprint / cooldown behavior.
8. Alert and work-order state models, ownership, history, audit, and conflict recovery work.
9. Unit, integration, frontend, security, and three end-to-end journeys pass as applicable.
10. Structured logs, four metrics, one trace, one measured load report, and one query-plan note are inspectable.
11. Main builds and deploys an identifiable release; public smoke and rollback guidance are current.
12. README, architecture, decisions, threat model, runbook, evidence index, screenshots, and demonstration are complete.
13. v1.0 is tagged and release notes list evidence plus material free-tier limitations.

Truthful release rule

Do not claim a feature, public deployment, security property, scale, performance result, or test count before corresponding implementation and evidence exist. If a free tier becomes unavailable or requires payment, select another no-card platform or publish the app as a local reproducible demonstration while clearly marking the public criterion incomplete.

08 | ACCEPTANCE AND CHANGE CONTROL

Acceptance and change control

The release must be evaluable, reproducible, and explainable without hidden setup.

Acceptance package

- Public repository at the tagged v1.0 commit.
- Public zero-cost HTTPS deployment identified by commit or image digest.
- Clearly sequenced clean-clone startup, green CI, and direct evidence links.
- Seeded demo guidance, bounded reset procedure, current screenshots, and ninety-second demonstration.
- Known limitations and post-v1 backlog.

Change process

Change	Required action
Clarification	Record it in the relevant issue or decision without expanding behavior.
Defect	Add a failing test where practical, correct behavior, and record evidence.
New v1 capability	Reject unless it closes an existing acceptance criterion.
Post-v1 idea	Record in backlog with rationale; do not partially implement it.
Schedule slip	Use the cut order; never hide an incomplete criterion.
Hosting policy change	Keep the no-payment constraint, update the deployment plan, and rerun public smoke before claiming readiness.

Priority under pressure

- Preserve tenant isolation, backend authorization, and truthful release status.
- Preserve atomic / idempotent telemetry, durable retry, and concurrency conflict.
- Preserve public zero-cost deployment, critical tests, and reproducible startup.
- Reduce responsive polish, charts, filters, and non-critical component breadth.
- Reduce operational UI or infrastructure automation while documenting manual gaps.

Hosting references checked for this revision

- Render free services and limits: https://render.com/docs/free
- Render Blueprint schema: https://render.com/docs/blueprint-spec
- Render no-payment deployment guide: https://render.com/docs/your-first-deploy
- Neon Free plan and current quotas: https://neon.com/pricing

Deliver the smallest complete release described by this contract. Completeness, correctness, inspectable evidence, zero-cost access, and clear trade-offs take precedence over additional features.